Code-Layout, Readability and Reusability

Date	30 September 2026
Team ID	SWTID-2026-4536
Project Name	LegalEase – AI-Powered Legal Document Generator
Maximum Marks	5 Marks

Code Layout Checklist

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing and indentation are used throughout the code.	Yes	Python code follows consistent indentation and formatting.
2	Proper File Structure	Files and folders are logically organized.	Yes	Frontend, backend, AI core, and document formatting modules are separated properly.
3	Meaningful Variable Names	Variables clearly reflect their purpose.	Yes	Names such as document_type, parties, terms, and dates clearly describe their purpose.
4	Function / Method Names	Functions are descriptively named.	Yes	Functions such as generate_document(), format_docx(), format_pdf(), and sanitize_text() are clearly named.
5	Code Comments	Inline and block comments explain important sections of code.	Yes	Comments are used to explain important backend, AI integration, and document formatting logic.

6	Modular Design	Code is split into reusable functions/modules.	Yes	FastAPI routes, Gemini AI integration, frontend, and document formatting are separated into modules.
7	No Redundant Code	Duplicate or unused code is removed.	Yes	Common document generation and formatting operations are implemented through reusable functions.
8	Error Handling	Exceptions and errors are handled gracefully.	Yes	API requests and document generation processes include error handling to prevent application failure.

Reusable Components / Modules

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level	
1	main.py	Python / FastAPI	Backend application initialization and API routing	High	
2	routes.py	Python / FastAPI	Document generation API requests	High	
3	gemini_generator.py	Python / Google Gemini API	AI-powered legal document generation	High	
4	format_docx()	Python / python-docx	DOCX generation for different legal documents	High	

5	format_pdf()	Python / FPDF	PDF generation for different legal documents	High	
6	format_html_preview()	Python / HTML / Streamlit	Previewing generated legal documents	High	
7	sanitize_text()	Python	Cleaning AI-generated text before formatting/export	High	
8	app.py	Python / Streamlit	User interface, editing and document downloads	High	

Overall Code Quality Assessment

Aspect	**Rating (1–5)**	**Comments**	
Code Layout & Structure	5	The project separates frontend, backend, AI integration, and document formatting into logical modules.	
Readability	5	Meaningful names, consistent formatting, and clear modular functions make the code easy to understand.	
Reusability	5	Document generation, formatting, sanitization, and preview functions can be reused for multiple legal document types.	
Documentation / Comments	4	Important modules and functions are documented to explain their purpose and usage.	
Overall Score	**5 / 5**	LegalEase follows a structured, readable, and modular development	

		approach suitable for future extension.	